

Claude Code の使い方

GitHub Copilot から乗り換える人のための入門ハンズオン

2026年6月 社内勉強会（60分）

今日のゴール: 帰るときに「できる」状態にする

✓ できるようになること	どこで扱うか
Claude Code が Copilot と何が違うか説明できる	背景編（前半15分）
定額プランの正体と「ボーナスタイム」がわかる	背景編
VS Code にインストールして最初のタスクを頼める	使い方・移行編（メイン35分）
Copilot との差異・つまずきポイントを回避できる	使い方・移行編
skill / customAgent / hooks のハーネス資産を両対応にできる	使い方・移行編
定額のうちにトークン感覚を鍛える理由がわかる	これから編（10分）

構成: ① 背景編 → ② 使い方・移行編（ここがメイン） → ③ これから編

① 背景編

そもそも何者で、なぜ乗り換えるのか（15分）

Claude Code とは — 「答える」ではなく「やり遂げる」

Anthropic 製の自律型 AI コーディングエージェント (2025年2月登場)

あなた: 「ログイン後に500になるバグ、直しといて」

Claude Code: リポジトリ探索 → 原因特定 → 修正 → **テスト実行** → 落ちたら自分で直して再実行
→ 結果報告

- 中身は「エージェントィックループ」: **状況把握** → **行動** → **検証** をゴールまで繰り返す
- 道具: ファイル読み書き / コード検索 / **シェル実行(テスト・ビルド・git)** / Web 検索
- シェルが使えるから **「動くことを確認済み」**で返してくる — 補完 AI には構造的に不可能

3 → 18%

職場採用率の急成長
(2025/4→2026/1)

46%

経験10年+の開発者の
第一choice (Copilotは9%)

80.8%

SWE-bench Verified
実Issue自動解決率

作っているのは Anthropic — OpenAI からの独立組

- 2021年、OpenAI の安全性研究の中核メンバーが独立して設立
 - Dario Amodei (元 OpenAI 研究VP、GPT-3 開発主導)、Daniela Amodei ら
 - 「商業化スピード vs 安全性」の路線対立が背景 → **AI セーフティが企業理念の中心**
 - 実利: **Team/Enterprise の入力データはモデル学習に不使用**
- ChatGPT/Codex (OpenAI) と Claude (Anthropic) は**同根のライバル**

⚠ 「Copilot で Claude を選ぶ」のと「Claude Code」は別物

	Copilot のモデルピッカーで Claude	Claude Code
モデル	Claude (同じ)	Claude (同じ)
ハーネス (実行環境)	GitHub 製	Anthropic 製・モデルと共進化

エージェント性能は**モデル × ハーネス**で決まる。純正フルチューンと載せ替えエンジンの差。

なぜ今乗り換えるか — 定額はどこに残っている？

2026年6月1日、Copilot は AI Credits (1クレジット=\$0.01) の従量制に移行。
補完は無料のままだが、Chat / Agent mode / CLI / レビュー = 全部クレジット消費。
1タスク \$1〜5 → 「今月あと何回頼めるか」を数えながら使う日々 🤖

プラン帯	Copilot	Claude Code	Codex
個人	従量 (2026/6〜)	✅ 定額 (Pro \$20 / Max \$100〜200)	✅ 定額 (Plus \$20 / Pro \$100〜)
Team / Business	従量 (Credits)	✅ 定額 Standard \$25〜30/席 (枠小) ✅ 定額 Premium \$100〜125/席	✅ 定額 Business \$25〜30/席 (枠小)
Enterprise	従量	従量 (pay-as-you-go)	従量

- Copilot だけが全プラン帯で従量。Claude / Codex は個人〜Team/Business に定額が残る（エンプラは両社とも従量）→ だから定額帯を選ぶ
- Claude Team は Standard（安いがトークン枠小）と Premium の2種。Codex Business も安い枠小
- エージェントをフル活用する前提なら枠の小さい席では足りない → うちの Claude Team Premium

「定額」の正体 — 上限・週次リミット・ボーナスタイム

Team Premium の定額は無制限ではない。正直に言うと:

- 5時間ごとにリセットされるセッション枠 + Weekly Limit (週次上限) の二段構え
- 上限に当たっても → 待てば回復、追加請求は1円も発生しない (ここが従量制との決定的な差)

 さらに、たまに「Limit リセット」が降ってくる

- Anthropic / OpenAI は新モデルのリリース記念や障害のお詫びで、全ユーザーの Limit をリセットすることがある
- 従量制では絶対に起きない、定額プランだけのボーナスタイム — 来たら遠慮なく使い倒す日

「失敗してもお金が減らない」は行動を変える。

雑に・大量に・実験的に頼める → 試行回数が稼げる → 上達する。

② 使い方・移行編

インストール、Copilot との差異、ハーネス資産の引っ越し（35分）

実行方法は4つ（中身は同じエージェント）

入口	起動方法	向いている用途
VS Code / JetBrains 拡張	拡張インストール	Copilot Chat 民はまずここ。差分をエディタで確認
CLI（基本形）	ターミナルで <code>claude</code>	すべての基本。SSH 先・コンテナ・サーバでも動く
デスクトップアプリ	Claude Desktop	複数エージェントの並列セッション管理
Web / クラウド	claude.ai/code	ローカル不要。クラウドサンドボックスで実行

ほか: GitHub Actions（PR で `@claude` メンション）、ヘッドレス `claude -p "..."`（CI から）

⚠ この2つは API 従量の別料金（定額シートの枠外） — CI に組み込むときは予算に注意

今日のおすすめ: VS Code 拡張で始めて、慣れたら CLI に降りる

CLI を覚える価値 — 操作体系が「業界標準」になっている

```
curl -fsSL https://claude.ai/install.sh | bash # または npm i -g @anthropic-ai/claude-code  
cd your-repo && claude # 初回はブラウザ認証
```

Claude Code CLI は各社にコピーされた「原型」

- Copilot CLI も Codex CLI も Devin CLI も Antigravity CLI も、操作体系は Claude Code CLI の後追い
 - REPL + スラッシュコマンド、`/init` 相当、`@` ファイル参照、許可モード… コマンド互換性が大きい
- つまり CLI で覚えた操作はツールを乗り換えても、ほぼそのまま通用する
 - GUI はツールごとに画面も概念も違って学び直し。CLI なら移行初日から同じ手が動く
 - 今回 Copilot → Claude の移行で苦労する人も、CLI に慣れれば次の乗り換えはタダ同然
- おまけ: SSH 先・コンテナ・CI でも同じ体験。 `claude -p "..."` でスクリプトからも呼べる (※API 従量・別料金)

🔧 ハンズオン1: VS Code へのインストール

5分で Copilot Chat の隣に Claude Code が並ぶ:

1. 拡張ビュー (`Cmd+Shift+X`) で 「**Claude Code**」を検索 (発行元: Anthropic) → Install
2. サイドバーの Claude アイコン → **Sign in** (会社の Team アカウントで認証)
3. 開き方は3通り: サイドバーアイコン / `Cmd+Esc` (Win: `Ctrl+Esc`) / コマンドパレット

Copilot Chat と同じ操作感

- チャットパネルに日本語で依頼
- `@` でファイル・フォルダを添付
- 変更は**エディタにインライン diff** → 承認/拒否

Copilot Chat に無いもの

- タブを増やして**複数セッション並列**
- 開いているファイル・選択範囲を自動認識
- コマンドパレットで plan モード切替
- 設定・履歴は CLI と共有 (`~/.claude`)

✅ 動作確認: 「このリポジトリの構成を説明して」と聞いてみる

Copilot Chat から来た人がつまづくポイント

つまづき	どうする
補完が出ない	仕様。補完は Copilot 併用（無料のまま）
<code>Shift+Tab</code> が効かない（VS Code）	インデント解除と競合。 <code>Cmd+Shift+P</code> → "Switch permission mode"
いちいち実行許可を聞かれる	ask がデフォルト。許可リスト（ <code>.claude/settings.json</code> ）を育てて auto へ
<code>copilot-instructions.md</code> が無視される	Claude が読むのは CLAUDE.md 。次のスライドで移行
長く使うと遅く・鈍くなる	1セッション=1タスク文化。区切りで <code>/clear</code> — 消しても <code>/resume</code> で過去セッションに復帰できる（VS Code でも可）
勝手に動いて怖い	plan モード で計画だけ出させて、承認してから実装させる

Copilot Chat は「会話」、Claude Code は「タスク依頼」。

ゴール・制約・完了の確認方法（テストコマンド）を渡すと、自分で検証してから報告してくる。

ライブデモ

`/init` → plan モードで実バグの修正を依頼 → 計画レビュー → 実装

見てほしいポイント:

- ①計画を立ててから動く
- ②自分でテストを回す
- ③落ちたら自分でリトライ

🔧 ハンズオン2: ハーネス資産の移行マップ

みんなが作り込んだ Copilot ハーネスには、**全部に対応物がある**:

Copilot で育てた資産	Claude Code での対応物	移行の手間
<code>copilot-instructions.md</code>	<code>CLAUDE.md</code>	<code>/init</code> + 統合を Claude に依頼
パス別 <code>*.instructions.md</code>	サブディレクトリ別 <code>CLAUDE.md</code>	ほぼ機械的
プロンプトファイル <code>*.prompt.md</code>	<code>.claude/commands/*.md</code>	ほぼ機械的
カスタムエージェント	<code>.claude/agents/*.md</code> (サブエージェント)	プロンプト本文は流用可
<code>skill</code>	<code>.claude/skills/</code> (スキル)	ほぼそのまま
<code>hooks</code>	<code>.claude/settings.json</code> の <code>hooks</code>	イベント名の読み替えのみ
MCP 設定	<code>.mcp.json</code>	共通規格・コピーで OK

手順は「移行作業自体を Claude に頼む」:

「`.github/` 配下の Copilot 設定を読んで Claude Code 用に変換して。**実コードと矛盾する古い記述は変換せず一覧で報告して**」

移行の本質 — ハーネスを「ツール非依存」にする

`.github/` は消さない。両対応（マルチエージェント）repo にする:

```
repo/
├── .github/copilot-instructions.md  ← Copilot 用エントリ（薄く残す）
├── CLAUDE.md                       ← Claude Code 用エントリ
├── AGENTS.md                       ← Codex ほか各社が読む業界標準エントリ
├── docs/agents/                   ← ★知識の本体（規約・コマンド・禁止事項）
└── .claude/ .mcp.json             ← Claude 固有の設定
```

- 各エントリファイルは `docs/agents/` を参照するだけの「薄いアダプタ」にする
- 知識の本体が1箇所 → Copilot でも Claude でも Codex でも動く repo、二重メンテなし
- 次の乗り換え（必ず来る）が「エントリ1枚追加」で済む — これがこのリファクタの本質
- 今日 Claude 用に作る構成は、そのまま将来の Codex 移行にも効く

③ これから編

定額が終わる前に、学んでおくこと（10分）

⚠ 定額は「ボーナスタイム」かもしれない

業界は明らかに従量制へ向かっている:

時期	出来事
2026年4月	Codex がトークン従量課金へ移行
2026年4月	Anthropic も Pro から Claude Code を一時削除（24時間で撤回） — 予兆
2026年6月	Copilot が AI Credits 従量制へ移行 ← 今ここ

エージェントの消費トークンは年々増える（並列化・長時間化）。
定額モデルはベンダーにとって重くなる一方 — **いつ転換されてもおかしくない。**

定額の今は「練習し放題の期間」。

従量制になった日に、トークン効率の良い使い方が身についている人と、
青天井の請求に怯える人に分かれる。

トークンの使い方 — 今日から鍛える5つの習慣

習慣	理由
① タスクの区切りで <code>/clear</code>	長いコンテキストは高い・遅い・精度も落ちる。消しても <code>/resume</code> で戻れるから恐れない
② 巨大なログ・JSON を貼らない	パスを渡して「必要な部分だけ読んで」
③ 調査はサブエージェントへ	子が10万トークン読んでも親には結論だけ
④ モデルを使い分ける	軽作業は Haiku/Sonnet、難所だけ Opus (API 単価は数十倍違う)
⑤ <code>/context</code> を見る癖	「今何トークン載ってるか」 = 従量時代の燃費計

これは節約術ではなくコンテキストエンジニアリング — 「モデルに何を読ませるか」の設計技術。
コンテキストを制する者がエージェントの精度も制する (料金は副産物)。
つまり良い使い方とトークン効率と同じもの。

ツールの使い方ではなく「体系」を学ぶ

ツールは1年で入れ替わる（このプレゼン自体が乗り換えの話）。残るのは概念:

- コンテキストエンジニアリング — 何を読ませ、何を読ませないかの設計
- 検証可能性の設計 — テスト・型・lint が整った repo ほど AI は強く働く
→ コード品質への投資 = AI の性能への投資（2026年の新常識）
- ハーネスのツール非依存設計 — 今日の移行リファクタがまさにこれ
- 権限設計 — 何を自動許可し、何を人間が握るか
- エージェントマネジメント — 1人が複数エージェントを並列監督する働き方へ

開発者の役割は「コードを書く人」から

「仕様を定義し、検証を設計し、エージェントを監督する人」へ

帰ったらまず、自分のリポジトリで

/init

Q&A

Copilot は解約？ → 補完は無料枠で併用が現実解

コードは学習される？ → *Team* はデフォルトで学習に不使用（必ず会社アカウントで）

暴走しない？ → *ask* モード + 許可リスト + フック + サンドボックス

定額枠を使い切ったら？ → 5時間でリセット・追加請求なし。頻発するなら */clear* 習慣を見直すサイン